# *THE DEFENSE OF WESTEROS*

The winter is coming, and the Night's Watch is preparing their defense against the more than possible attack from the White Walkers, a one hundred thousand man army that lurks in the darkness. To prevent it, Lord Commander has entrasted us with analyzing images that our ravens get from beyond the wall. The goal is to analyze these images to find and detect enemies and possible threats by implementing histograms, look-up-tables and different techniques for image processing that we are going to explain further in this notebook.



```
try:
    from google.colab import drive
    drive.mount('/content/drive')
    # If your notebook and images are in your Drive, change directory accordingly:
    %cd '/content/drive/Othercomputers/Mi portátil/cuadernos_extra'
except ImportError:
    pass

import numpy as np
from scipy import signal
import cv2
import matplotlib.pyplot as plt
import matplotlib
from ipywidgets import interactive, fixed, widgets
matplotlib.rcParams['figure.figsize'] = (15.0, 15.0)
import random

images_path = './images/'
```

```
Mounted at /content/drive
/content/drive/Othercomputers/Mi portátil/cuadernos_extra
```

## *CHAPTER I: THE NIGHT AT WINTERFELL*

For this first part, we are going to inspect the images from our ravens and try to improve the global contrast in order to make our job easier. This will involve analyzing the pixel distribution through histograms to understand the current state of the image and then applying techniques to spread out these pixel intensities, enhancing visibility of critical details that might otherwise be hidden in the low-contrast blizzard conditions.

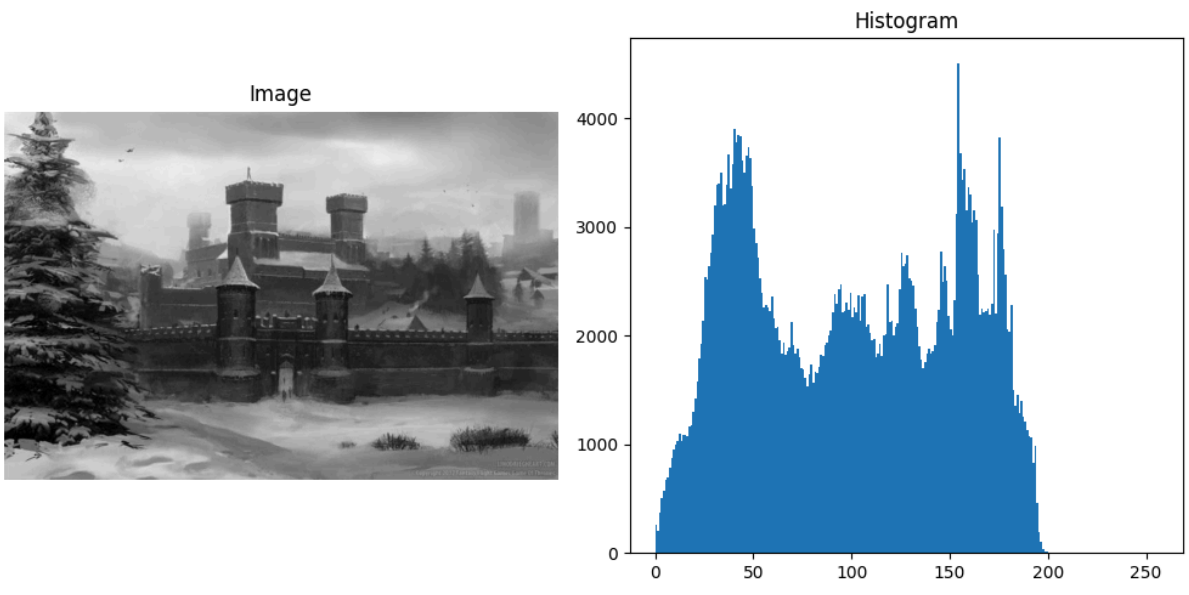
```
image = cv2.imread(images_path + 'winterfell_blizzard.jpg', cv2.IMREAD_GRAYSCALE)

plt.figure(figsize=(10, 5)) # Increased figure size
plt.subplots_adjust(wspace=0.3) # Add spacing between subplots

plt.subplot(1,2,1)
plt.title("Image")
plt.imshow(image, cmap='gray')
plt.axis('off') # Hide axes for cleaner image display

# Now, show its histogram
plt.subplot(1,2,2)
plt.title("Histogram")
plt.hist(image.ravel(), 256, range=[0, 256])

plt.tight_layout() # Adjust layout to prevent overlapping
plt.show()
```




By seeing the histogram of our picture, we can determine that the image has a big concentration of pixels between 0 and 200, but there are no pixels higher than 200. This indicates a low-contrast image, especially in the brighter regions, where details might be lost due to the compressed range of pixel intensities. To effectively identify threats and features in these challenging conditions, we need to enhance the contrast, which histogram equalization can help achieve by distributing the pixel values across the full dynamic range.

### HISTOGRAM EQUALIZATION

After the night watch of the ravens, they bring us completely dark images, where, in addition, the snow and fog make it difficult for us to detect enemies. For this, we are going to use histogram equalization to differentiate them. As we can see, in dark images, all pixels are concentrated in the range [0,30] of our histogram, making it impossible to see anything in the dark. With histogram equalization, we will make the pixel intensities more evenly distributed across the histogram, and thus be able to distinguish the silhouettes of the white walkers.

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

def equalize_chart(image, verbose=False):
    """ Applies histogram equalization to the luminance channel of an image in YCrCb space.
```

```
    Args:
        image (numpy.ndarray): Input image in BGR color space.
        verbose (bool): If True, plots the original and processed images along with their histograms.

    Returns:
        numpy.ndarray: Equalized image in BGR color space.
"""
# 1. Transformación al espacio de color YCrCb desde BGR
ycrcb_image = cv2.cvtColor(image, cv2.COLOR_BGR2YCrCb)

# 2. Ecualización del canal de luminancia (Y, canal 0)
ycrcb_image[:, :, 0] = cv2.equalizeHist(ycrcb_image[:, :, 0])

# 3. Reconversión al espacio de color BGR original para el retorno
out_image = cv2.cvtColor(ycrcb_image, cv2.COLOR_YCrCb2BGR)

# 4. Bloque de visualización (separado del procesamiento principal)
if verbose:
    plt.figure(figsize=(10, 8))

    # Conversión a RGB exclusivamente para una visualización correcta en Matplotlib
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    out_image_rgb = cv2.cvtColor(out_image, cv2.COLOR_BGR2RGB)

    # Muestra de la imagen original
    plt.subplot(2, 2, 1)
    plt.imshow(image_rgb)
    plt.title('Original image')
    plt.axis('off')

    # Muestra de la imagen procesada
    plt.subplot(2, 2, 2)
    plt.imshow(out_image_rgb)
    plt.title('Equalized histogram image')
    plt.axis('off')

    # Histograma original (usando el canal Y de luminancia)
    orig_y = cv2.cvtColor(image, cv2.COLOR_BGR2YCrCb)[:, :, 0]
    plt.subplot(2, 2, 3)
    plt.hist(orig_y.ravel(), bins=256, range=[0, 256])
    plt.title('Original histogram (Y channel)')

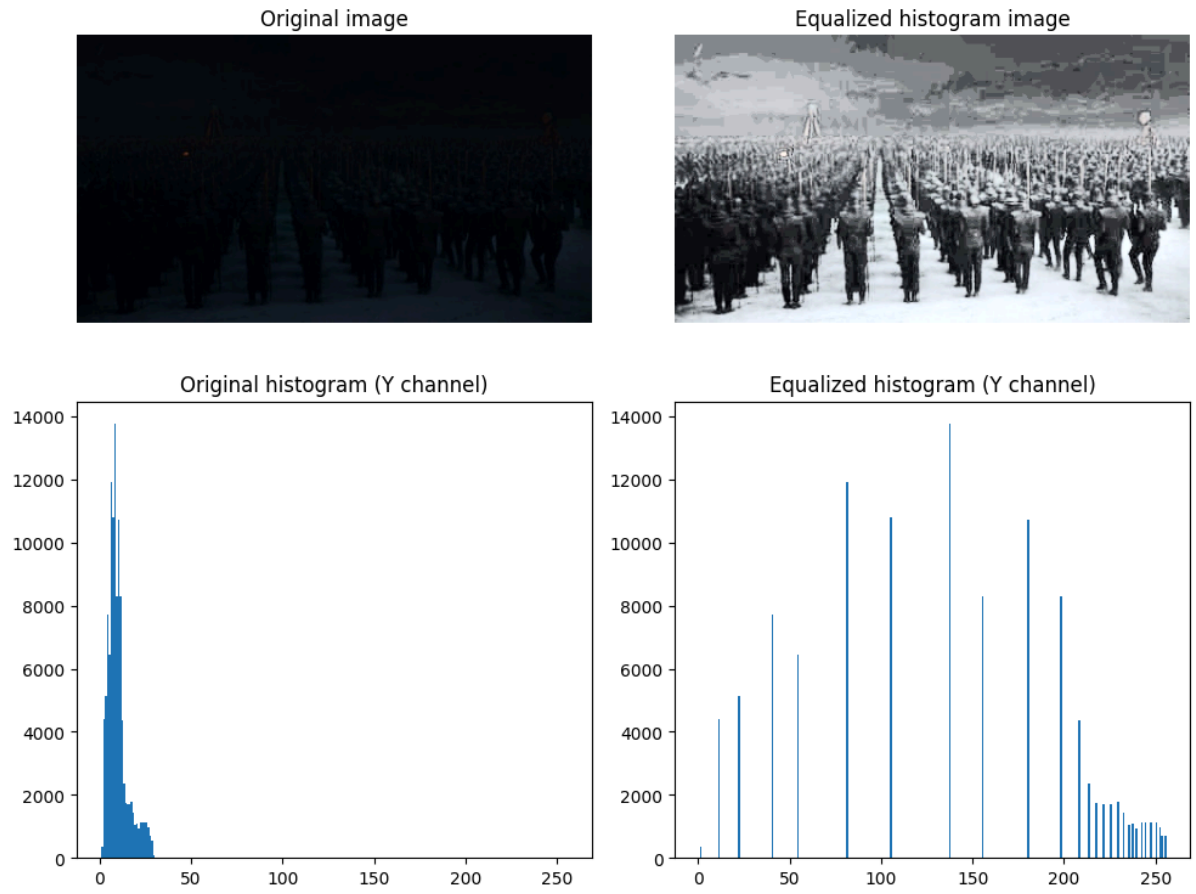
    # Histograma ecualizado
    plt.subplot(2, 2, 4)
    plt.hist(ycrcb_image[:, :, 0].ravel(), bins=256, range=[0, 256])
    plt.title('Equalized histogram (Y channel)')

    plt.tight_layout()
    plt.show()

return out_image
```

```
# Read image
image = cv2.imread(images_path + 'batalla.jpg',-1)

# Equalize its histogram
interactive(equalize_chart, image=fixed(image), verbose=fixed(True))
```




Thanks to the incredible power of histogram equalization, we've boosted the contrast, revealing the enemy army in all its glorious gloom. No more squinting into the blizzard, Lord Commander! Those White Walkers thought they were being clever, but our pixel-manipulation prowess has exposed their icy faces. Now we know exactly who's coming for tea (and a battle!), and we can finally tell if that's a direwolf or just a really fluffy snowdrift. Victory is practically in the bag!



## SMOOTHING

Even after enhancing contrast with histogram equalization, images can suffer from noise, which obscures important details. To address this, we can apply smoothing techniques like the Gaussian filter. Smoothing helps to reduce random variations in image intensity, making features more discernible without losing too much detail. This initial step of noise reduction can significantly improve the quality of subsequent image processing operations.

```
def gaussian_filter(image, w_kernel, sigma, verbose=False):
    """ Applies Gaussian filter to an image and display it.

        Args:
            image: Input image
            w_kernel: Kernel aperture size
            sigma: standard deviation of Gaussian distribution
            verbose: Only show images if this is True

        Returns:
            smoothed_img: smoothed image
    """
    # Write your code here!

    # Create kernel using separability property
    s = sigma
    w = w_kernel
    kernel_1D = np.float32([1 / (s * np.sqrt(2 * np.pi)) * np.exp(- (z**2) / (2 * s**2)) for z in range(-w,w+1)]) # Evaluate the gaussian in "expression"
    vertical_kernel = kernel_1D.reshape(2*w+1,1) # Reshape it as a matrix with just one column
    horizontal_kernel = kernel_1D.reshape(1,2*w+1) # Reshape it as a matrix with just one row
    kernel = signal.convolve2d(vertical_kernel, horizontal_kernel) # Get the 2D kernel

    # Convolve image and kernel
    smoothed_img = cv2.filter2D(image,-1,kernel)

    if verbose:
        plt.figure(figsize=(10, 5)) # Set a good figure size
        # Show the initial image
        plt.subplot(121)
        plt.imshow(image, cmap='gray')
        plt.title('Noisy')
        plt.axis("off")

        # Show the resultant one
        plt.subplot(122)
        plt.imshow(smoothed_img, cmap='gray')
        plt.title('Gaussian filter')
        plt.axis("off")
        plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
        plt.show() # Display the plots

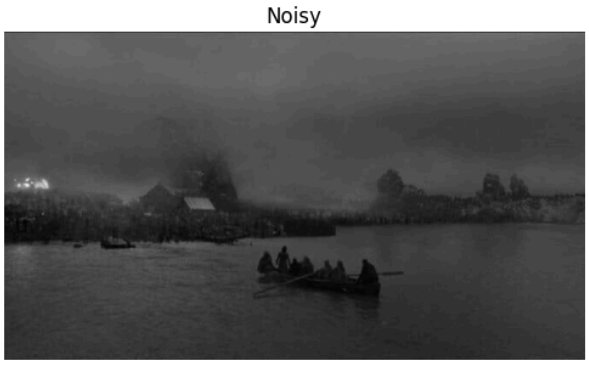
    return smoothed_img
```

```
# Interact with the kernel size and the sigma value
noisy_img = cv2.imread(images_path + 'smoothing_fog.jpeg', 0)
interactive(gaussian_filter, image=fixed(noisy_img), w_kernel=(0,5,1), sigma=(1,3,0.3), verbose=fixed(True))
```

w_kernel 2

sigma 1.90

Noisy



Gaussian filter



## CLAHE

After addressing general image noise with smoothing, we can now refine the contrast enhancement using more advanced techniques. While global histogram equalization can introduce artifacts, Contrast Limited Adaptive Histogram Equalization (CLAHE) offers a more localized approach. CLAHE divides the image into small regions, or 'tiles,' and performs equalization on each one, limiting the contrast to avoid over-amplification of noise. This method provides a better quality picture by enhancing contrast adaptively, making details clearer, especially in areas that might still be too dark or too bright.

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
import ipywidgets as widgets
from ipywidgets import interactive, fixed

def clahe_chart(image, clip_limit=2.0, tile_grid_size=(8,8), verbose=False):
    """ Applies CLAHE to an image and shows the result.

        Args:
            image: Input image
            clip_limit: Threshold for contrast limiting
            tile_grid_size: Size of grid for histogram equalization
            verbose: Only show images if this is True

        Returns:
            out_image: CLAHE enhanced image
"""

# Transform image to YCrCb color space
image = cv2.cvtColor(image, cv2.COLOR_BGR2YCrCb)
out_image = np.copy(image)

# Create CLAHE object
clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_grid_size)

# Apply CLAHE to first band of the YCrCb image
out_image[:,:,0] = clahe.apply(out_image[:,:,0])

if verbose:
    plt.figure(figsize=(15, 10)) # Increased figure size for better visibility

    # Apply global histogram equalization for comparison
    equalized_image = np.copy(image)
    equalized_image[:,:,0] = cv2.equalizeHist(equalized_image[:,:,0])

    # Plot histograms
    plt.subplot(2,3,4)
    plt.hist(image[:,:,0].ravel(), 256, range=[0, 256])
    plt.title('Original histogram')

    plt.subplot(2,3,5)
    plt.hist(equalized_image[:,:,0].ravel(), 256, range=[0, 256])
    plt.title('Equalized histogram')

    plt.subplot(2,3,6)
    plt.hist(out_image[:,:,0].ravel(), 256, range=[0, 256])
    plt.title('CLAHE histogram')

    # Reconvert images to RGB
    image = cv2.cvtColor(image, cv2.COLOR_YCrCb2RGB)
    equalized_image = cv2.cvtColor(equalized_image, cv2.COLOR_YCrCb2RGB)
    out_image = cv2.cvtColor(out_image, cv2.COLOR_YCrCb2RGB)

    # Show the images
```

```
        plt.subplot(2,3,1)
        plt.imshow(image)
        plt.title('Original Image')

        plt.subplot(2,3,2)
        plt.imshow(equalized_image)
        plt.title('Global equalization')

        plt.subplot(2,3,3)
        plt.imshow(out_image)
        plt.title('CLAHE')

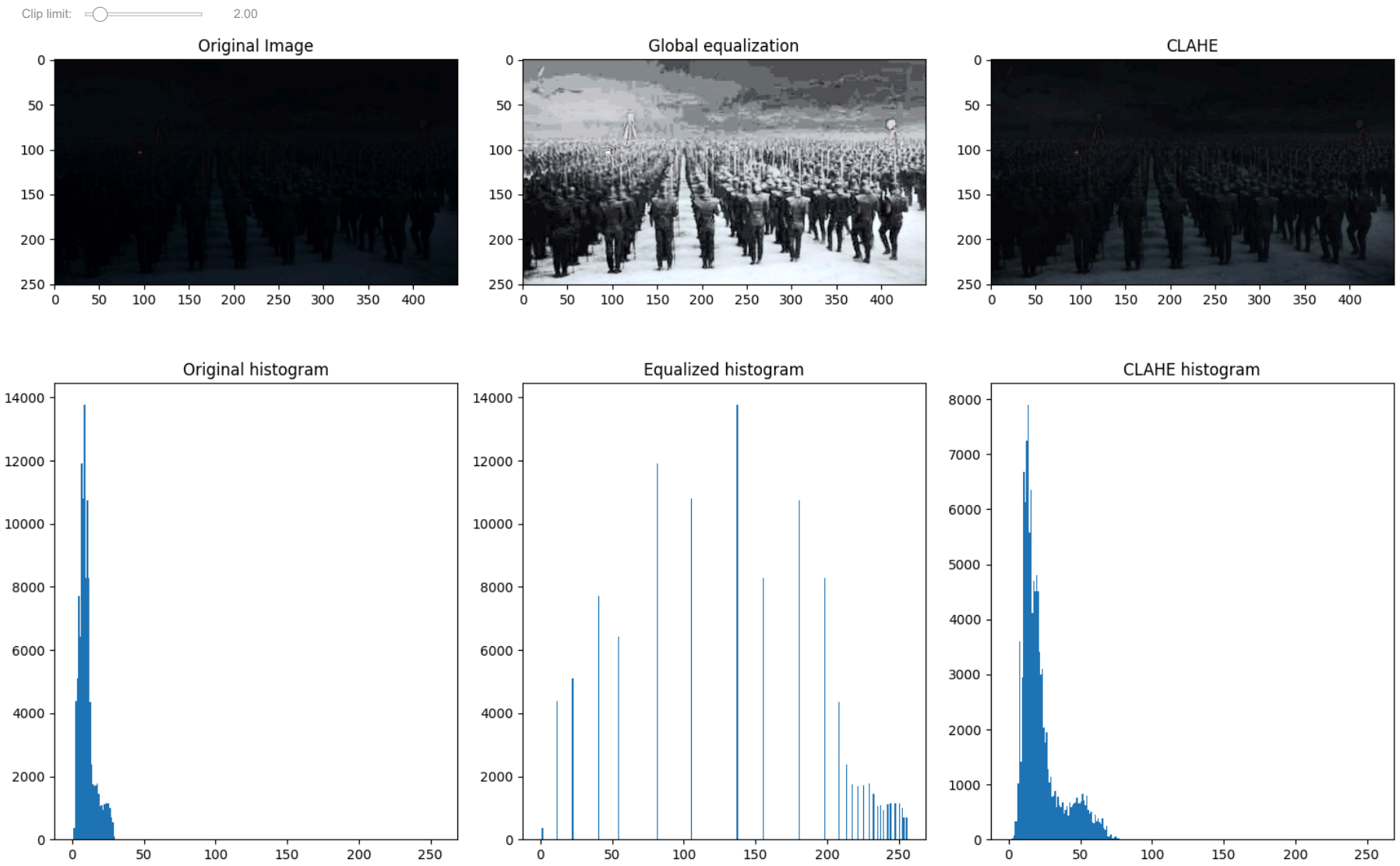
        plt.tight_layout()
        plt.show()

    return out_image
```

```
#Create widget object
clip_widget = widgets.FloatSlider(value=2.0, min=1.0, max=10.0, step=0.5, description='Clip limit:')

# Read image
image = cv2.imread(images_path + 'batalla.jpg')

# Interact with your code!
interactive(clahe_chart, image=fixed(image), clip_limit=clip_widget, tile_grid_size=fixed((8,8)), verbose=fixed(True))
```




Although we can see aour enemies, the equalized image presents noise in the background and we can get a better quality picture with another image enhancement techniques, such as CLAHE(Contrast Limited Adaptive Histogram Equalization).Clahe will let us limit contrast enhancement and avoid noise amplification We are going to divide the image into small regions called tiles and we are going to equalize each tile locally using a **clipLimit** before calculating the cumulative distribution function (CDF).

## *CHAPTER II: THE EDGES OF THE LONG NIGHT*

The ravens keep sending us images from beyond the Wall. However, due to the heavy fog and thick snow, our scouts cannot easily distinguish where the background ends and an enemy begins. To mount an effective watch from the towers of Winterfell, we need an edge detection system to help us recognize enemies, weapons, or incoming siege structures.

### DIRECTIONAL EDGE DETECTION (SOBEL)

For this technique, we are going to work with an image of Winterfell. Edges correspond to sharp transitions in pixel intensity. A difference must be significant enough to be considered a true boundary.

There are several edge detection methods, but we will start with first-derivative operators. By convolving the image with Sobel kernels ( $G_x$ and $G_y$), we can calculate the horizontal and vertical gradients. This allows us to estimate the boundary directions and combine them to compute the total gradient magnitude.

```
def edge_detection_chart(image, kernel_h, kernel_v, threshold, verbose=False):
    """ Computed the gradient of the image, binarizes and display it.

        Args:
            image: Input image
            kernel_h: kernel for horizontal derivative
            kernel_v: kernel for vertical derivative
            threshold: threshold value for binarization
            verbose: Only show images if this is True

        Returns:
            edges: edges binary image
    """
    # Compute derivatives
    d_h = cv2.filter2D(image, cv2.CV_16S, kernel_h)
    d_v = cv2.filter2D(image, cv2.CV_16S, kernel_v)

    # Compute gradient
    gradient_image = np.add(np.absolute(d_h), np.absolute(d_v))

    # Normalize gradient
    norm_gradient = cv2.normalize(gradient_image, None, 0, 255, cv2.NORM_MINMAX)

    # Threshold to get edges
    ret, edges = cv2.threshold(norm_gradient, threshold, 255, cv2.THRESH_BINARY)

    if verbose:
```

```
        # Set dark style and adjust figure size
        with plt.style.context('dark_background'):
            plt.figure(figsize=(15, 5))

            # Show the initial image
            plt.subplot(131)
            plt.imshow(image, cmap='gray')
            plt.title('Original image')
            plt.axis('off')

            # Show the gradient image
            plt.subplot(132)
            plt.imshow(gradient_image, cmap='gray')
            plt.title('Gradient image')
            plt.axis('off')

            # Show edges image
            plt.subplot(133)
            plt.imshow(edges, cmap='gray')
            plt.title('Edges detected')
            plt.axis('off')

            plt.tight_layout()
            plt.show()

    return edges
```

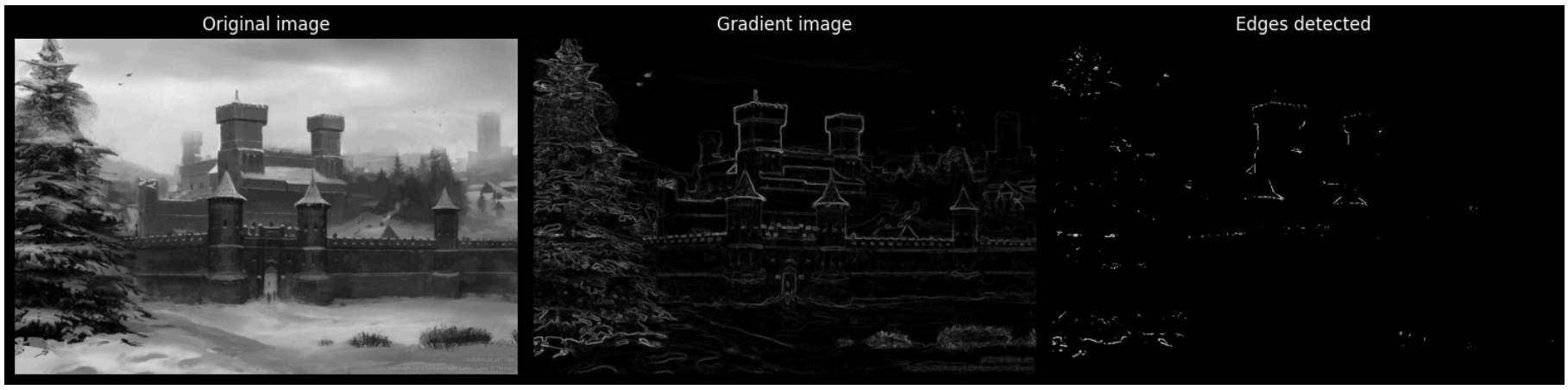
```
# Read image
image = cv2.imread(images_path + 'winterfell_blizzard.jpg', 0)

# Define kernel (Sobel)

kernel_h = np.array([[1,0,-1],[2,0,-2],[1,0,-1]])*1/4
kernel_v = np.array([[-1,-2,-1],[0,0,0],[1,2,1]])*1/4

interactive( edge_detection_chart, image=fixed(image), kernel_h=fixed(kernel_h), kernel_v=fixed(kernel_v), threshold=(0,255,10), verbose=fixed(True))
```

threshold 120




As we can see in the original image, the edges transition from dark pixels to brighter ones. This is the reason why we need to use a lower threshold. In this case, a value of 30 works well because it allows us to detect the outlines of the structures and terrain perfectly.

The Lookout's Dilemma: Single-Threshold Edge Detection

## CANNY EDGE DETECTION

To overcome the "Lookout's Dilemma" of using a single global threshold, we are going to use the Canny Edge detection, with two thresholds.

The Canny detector consists of four main steps:

1. **Noise Reduction**: Applies a 5x5 Gaussian filter to smooth the image and remove noise.
2. **Intensity Gradient**: Calculates the horizontal and vertical derivatives to find the gradient magnitude and direction.
3. **Non-Maximum Suppression**: Thins out the edges by suppressing pixels that are not local maxima in the direction of the gradient.
4. **Hysteresis Thresholding**: Uses two threshold values (`threshold1` and `threshold2`):
    - Pixels with a gradient intensity above `threshold2` are classified as strong edges (sure edges).
    - Pixels below `threshold1` are discarded.
    - Pixels between these two thresholds are only kept if they are connected to a strong edge.

```
def canny_edge_chart(image, threshold1, threshold2, verbose=False):
    """ Applies Canny edge detection and displays the result with a dark background style.
    """
    # Apply Canny algorithm
    edges = cv2.Canny(image, threshold1, threshold2)

    if verbose:
        with plt.style.context('dark_background'):
            plt.figure(figsize=(12, 6))

            plt.subplot(121)
            plt.imshow(image, cmap='gray')
            plt.title('Original Image')
            plt.axis('off')

            plt.subplot(122)
            plt.imshow(edges, cmap='gray')
            plt.title('Canny Edges (Hysteresis)')
            plt.axis('off')

            plt.tight_layout()
            plt.show()

    return edges
```

```
# Read the same winterfell image
image_winterfell = cv2.imread(images_path + 'winterfell_blizzard.jpg', 0)

# Set up interactive sliders for both thresholds
interactive(canny_edge_chart,
            image=fixed(image_winterfell),
            threshold1=widgets.IntSlider(value=50, min=0, max=255, step=5, description='Threshold 1 (Min):'),
            threshold2=widgets.IntSlider(value=150, min=0, max=255, step=5, description='Threshold 2 (Max):'),
            verbose=fixed(True))
```

Threshold … 50

Threshold … 150




## COMPARATIVE CONCLUSION

While **Sobel** is a quick and simple method that detects basic transitions but is very sensitive to noise and generates thick, fragmented edges, the **Canny** algorithm offers a far superior result. By integrating a preliminary Gaussian filter to reduce noise, performing non-maximum suppression to thin edges, and applying a double threshold (hysteresis), Canny manages to trace clean, continuous, and precise contours without being affected by the chaos of the blizzard.

## *CHAPTER III: THE MAP OF THE DEFENSE*

The maesters of the Citadel and the strategists of Queen Daenerys Targaryen need a continuous and accurate map of the defenses of King's Landing. The dragonback scouts have taken several aerial perspectives: one at a high altitude that covers a wide but distant view, and another at a low altitude, with an inclined and rotated angle as they fly over the walls.

The goal is to find reliable matching points between both shots despite changes in scale, orientation, and lighting conditions, in order to estimate the geometric relationship between them.

### Image Transformation Process

To simulate the perspective change captured by the low-altitude dragon scout, we apply a geometric transformation to the original image (`ciudad.jpg`). This process involves rotating and scaling the image using the following OpenCV functions:

1. `cv2.getRotationMatrix2D(center, angle, scale)`: Computes the 2D affine transformation matrix. It takes the center point of the transformation, the rotation angle in degrees (positive for counter-clockwise), and an isotropic scale factor to resize the image.
2. `cv2.warpAffine(src, M, dsize)`: Applies the affine transformation calculated previously. It takes the source image, the $2 \times 3$ transformation matrix $M$, and the output image dimensions `dsize` to generate the final warped image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load the original city image
city_image = cv2.imread(images_path + 'ciudad.jpg')

# 2. Define the transformation parameters (e.g., rotation of 30 degrees and scale of 0.7)
height, width = city_image.shape[:2]
center = (width / 2, height / 2)
angle = 30
scale = 0.7

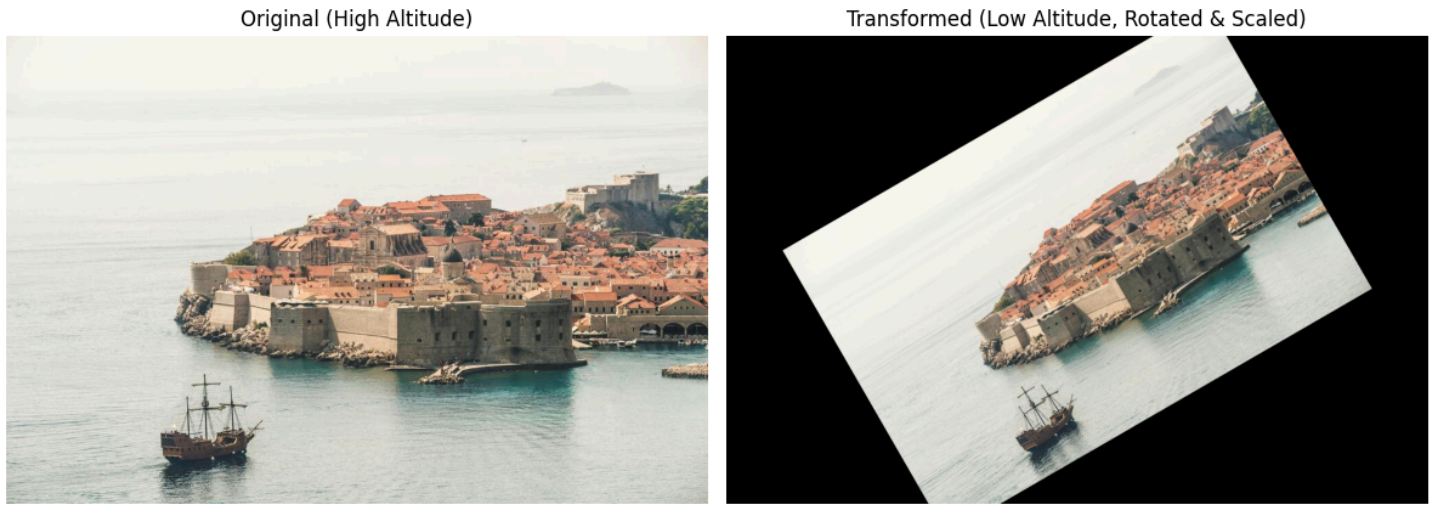
# 3. Compute the affine transformation matrix for rotation and scaling around the center
M = cv2.getRotationMatrix2D(center, angle, scale)

# 4. Apply the affine transformation to the original image
city_image2 = cv2.warpAffine(city_image, M, (width, height))

# Visualize both images side by side
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title("Original (High Altitude)")
plt.imshow(cv2.cvtColor(city_image, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Transformed (Low Altitude, Rotated & Scaled)")
plt.imshow(cv2.cvtColor(city_image2, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.tight_layout()
plt.show()
```




## HARRIS CORNER DETECTOR

Now, we are going to compare how the Harris Corner detector can detect corners in both images

```
def apply_harris(image, size_window, size_sobel, const_k):
    """
    Apply the Harris detector to an RGB image.
```

```
    """
    # Convert to gray
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray = np.float32(gray)

    # Apply Harris
    harris_response = cv2.cornerHarris(gray, size_window, size_sobel, const_k)

    return harris_response

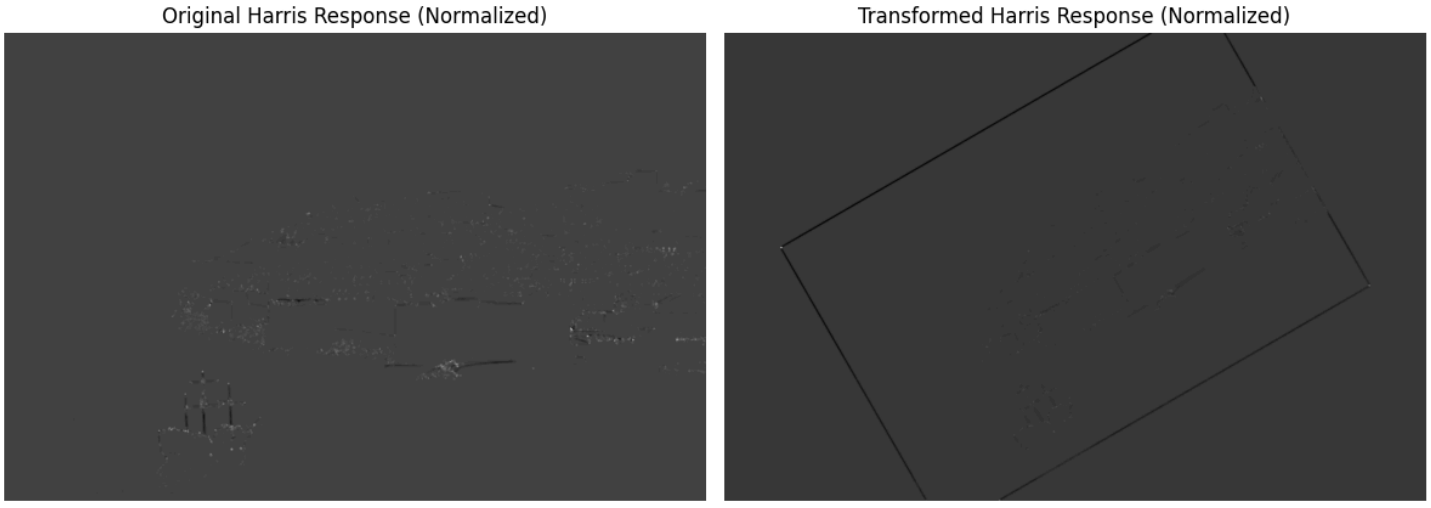

size_window = 3
size_sobel = 3
const_k = 0.04
harris1 = apply_harris(city_image, size_window, size_sobel, const_k)
harris2 = apply_harris(city_image2, size_window, size_sobel, const_k)

# Normalize the harris responses so they can be properly visualized in [0, 255]
harris1_norm = cv2.normalize(harris1, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)
harris2_norm = cv2.normalize(harris2, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)

plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title("Original Harris Response (Normalized)")
plt.imshow(harris1_norm, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Transformed Harris Response (Normalized)")
plt.imshow(harris2_norm, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```




## Visualizing Corners on top of the Original Images

To physically locate and identify the corners, we apply a threshold to the response map computed by the Harris detector. Pixels with a corner strength exceeding a specific percentage of the maximum response (`harris.max()`) are marked as valid corners. We then overlay these detected keypoints as bright red markers on top of our original color images.

```
# Define a threshold (e.g., 1% of the maximum response value of Harris)
threshold_percentage = 0.01

# Make a copy of the original color images to draw on top of them
city_corners1 = city_image.copy()
city_corners2 = city_image2.copy()

# Find the coordinates where the response exceeds the threshold and draw circles
# For the original image
city_corners1[harris1 > threshold_percentage * harris1.max()] = [0, 0, 255] # Draw in Red (BGR)

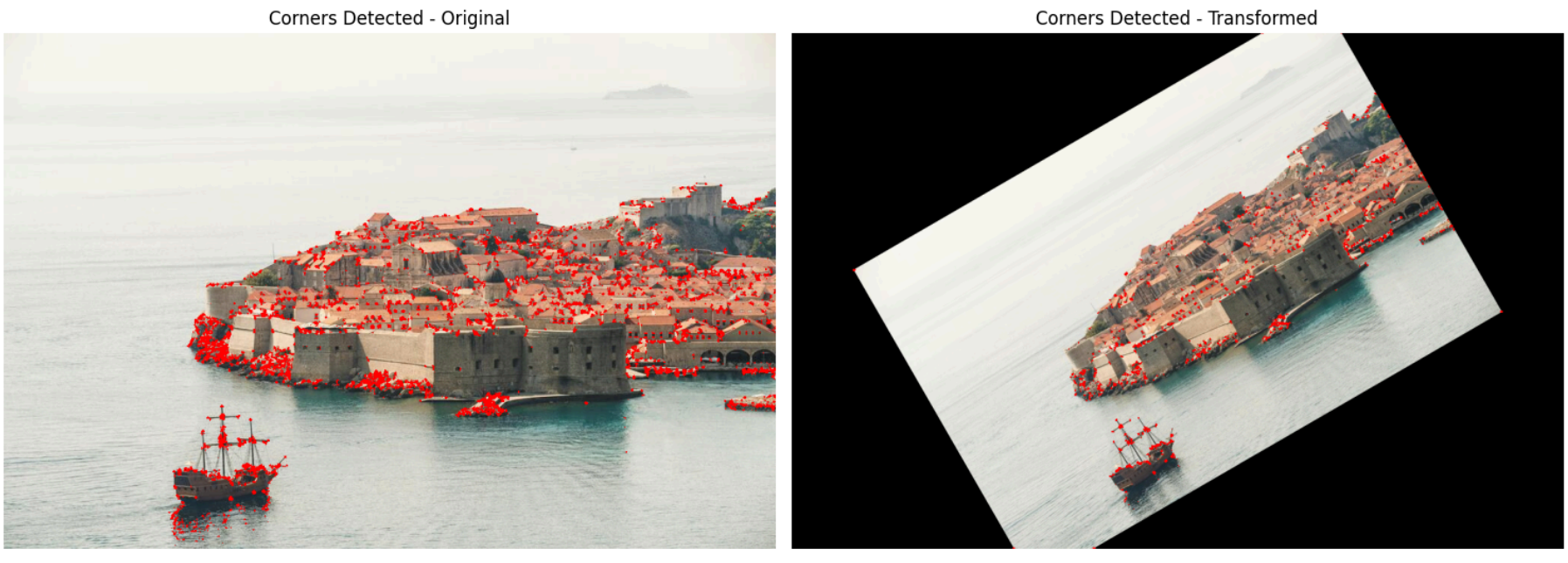
# For the transformed image
city_corners2[harris2 > threshold_percentage * harris2.max()] = [0, 0, 255] # Draw in Red (BGR)

# Visualize the results side-by-side
plt.figure(figsize=(15, 8))

plt.subplot(1, 2, 1)
plt.title("Corners Detected - Original")
plt.imshow(cv2.cvtColor(city_corners1, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Corners Detected - Transformed")
plt.imshow(cv2.cvtColor(city_corners2, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.tight_layout()
plt.show()
```




As we can see in both images and as we have learnt in the theory part of keypoints sections, Harris corner detector is invariant to rotation as R (single scalar response) depends on the eigenvalues (eigenvalues show how strong is the variation of intensity in two directions and "dont care" about the direction itself) that do not change during the rotation of the image. However, it is not invariant to scale, that is why there are aa lot of corners on the left image that are not correctly detected in the transformed one.

## TEMPLATE MATCHING VIA NORMALIZED CROSS CORRELATION (NCC)

While corner detectors like Harris find generic points of interest, Normalized Cross Correlation (NCC) is a technique used for template matching. It measures the degree of similarity between a small sub-image (template) and all possible patches of the target image as it slides across it.

Using the formula for normalized correlation coefficients, NCC provides a value in the range of `[-1, 1]`, where `1` represents a perfect match. This normalization makes the method robust to global changes in illumination (brightness and contrast).

```
# 1. Define a template patch from the original image (e.g., a 60x60 building area)
template_y, template_x = 300, 450
template_h, template_w = 60, 60
template = city_image[template_y:template_y+template_h, template_x:template_x+template_w]

# 2. Convert images to grayscale for template matching
gray_city = cv2.cvtColor(city_image, cv2.COLOR_BGR2GRAY)
gray_city2 = cv2.cvtColor(city_image2, cv2.COLOR_BGR2GRAY)
gray_template = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)

# 3. Apply Template Matching using Normalized Cross Correlation (TM_CCOEFF_NORMED)
ncc_map1 = cv2.matchTemplate(gray_city, gray_template, cv2.TM_CCOEFF_NORMED)
ncc_map2 = cv2.matchTemplate(gray_city2, gray_template, cv2.TM_CCOEFF_NORMED)

# 4. Find the location with the maximum correlation score
min_val1, max_val1, min_loc1, max_loc1 = cv2.minMaxLoc(ncc_map1)
min_val2, max_val2, min_loc2, max_loc2 = cv2.minMaxLoc(ncc_map2)

# 5. Draw bounding boxes on copies of our color images around the best matches
match_img1 = city_image.copy()
top_left1 = max_loc1
bottom_right1 = (top_left1[0] + template_w, top_left1[1] + template_h)
cv2.rectangle(match_img1, top_left1, bottom_right1, (0, 255, 0), 3) # Green box

match_img2 = city_image2.copy()
top_left2 = max_loc2
bottom_right2 = (top_left2[0] + template_w, top_left2[1] + template_h)
cv2.rectangle(match_img2, top_left2, bottom_right2, (0, 0, 255), 3) # Red box (potential mismatch due to rotation/scale)

# Plot results
plt.figure(figsize=(15, 8))

# Original image match
plt.subplot(1, 3, 1)
plt.title(f"Original Matching (NCC Max: {max_val1:.2f})")
plt.imshow(cv2.cvtColor(match_img1, cv2.COLOR_BGR2RGB))
plt.axis('off')

# Template being searched
plt.subplot(1, 3, 2)
plt.title("Template to Find")
plt.imshow(cv2.cvtColor(template, cv2.COLOR_BGR2RGB))
plt.axis('off')

# Transformed image match
plt.subplot(1, 3, 3)
plt.title(f"Transformed Matching (NCC Max: {max_val2:.2f})")
plt.imshow(cv2.cvtColor(match_img2, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.tight_layout()
plt.show()
```




- **In the Original Image:** The template is found perfectly with a maximum correlation score very close to `1.0`. Since the template was extracted from this image, the alignment matches down to the exact pixel coordinate.
- **In the Transformed Image:** Notice how the maximum correlation is 0.44, significantly lower and it shows the incorrect region of the image. This happens because NCC is not invariant to scale transformation or rotation

## SIFT (SCALE-INVARIANT FEATURE TRANSFORM)

To overcome the structural vulnerabilities of NCC and classical pixel-level matching, we utilize the **Scale-Invariant Feature Transform (SIFT)**. SIFT detects distinctive keypoints in scale space and extracts invariant descriptor vectors for each keypoint. This ensures robust matching regardless of rotation, scaling, or 3D viewpoint changes.

We will now use SIFT to detect, compute, and match feature correspondences between the high-altitude original view and the low-altitude transformed view of King's Landing.

```
# 1. Initialize SIFT detector
sift = cv2.SIFT_create()

# 2. Find the keypoints and descriptors with SIFT
kp1, des1 = sift.detectAndCompute(gray_city, None)
kp2, des2 = sift.detectAndCompute(gray_city2, None)

# 3. Use FLANN matcher to find the matching points
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)

matches = flann.knnMatch(des1, des2, k=2)

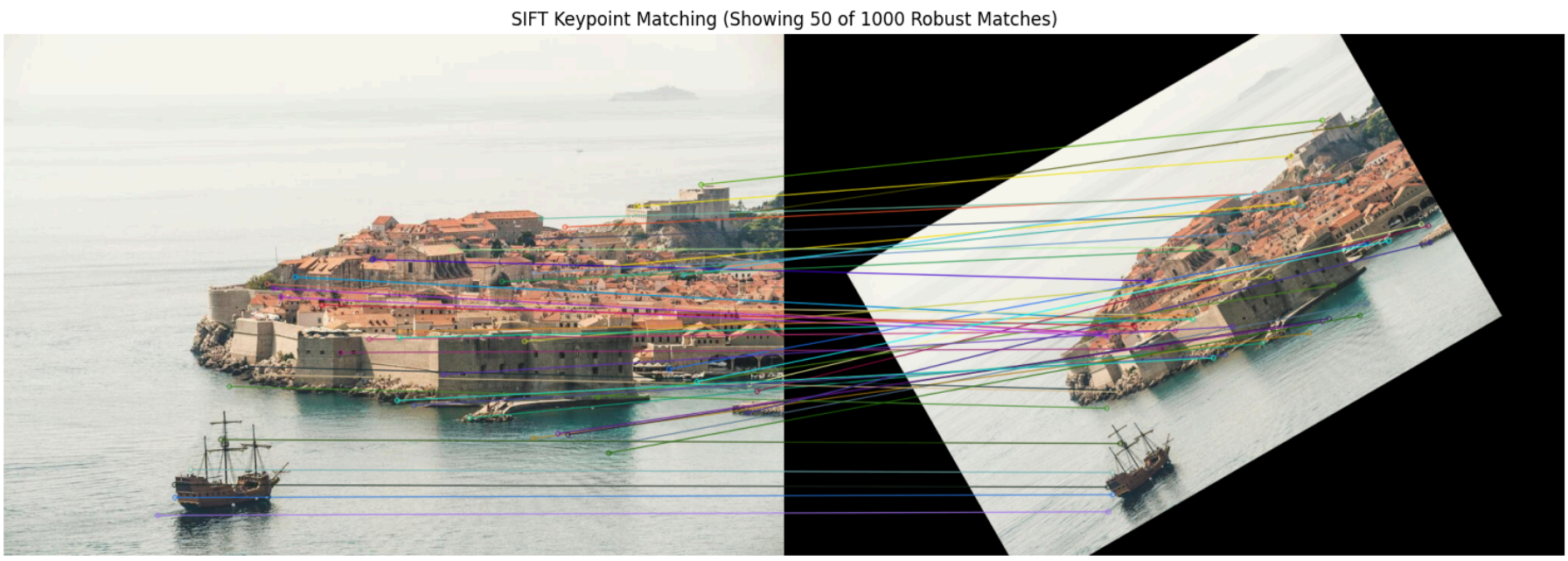
# 4. Filter matches using Lowe's ratio test
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

# Sort matches based on their distance so we can display the best ones first
good_matches = sorted(good_matches, key=lambda x: x.distance)
```

```
# Select a subset of the best matches to avoid cluttering the visualization (e.g., top 50)
matches_to_show = good_matches[:50]

# 5. Draw matches
sift_matches_img = cv2.drawMatches(
    city_image, kp1,
    city_image2, kp2,
    matches_to_show, None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# Plot results
plt.figure(figsize=(15, 10))
plt.title(f"SIFT Keypoint Matching (Showing {len(matches_to_show)} of {len(good_matches)} Robust Matches)")
plt.imshow(cv2.cvtColor(sift_matches_img, cv2.COLOR_BGR2RGB))
plt.axis('off')
plt.tight_layout()
plt.show()
```




- Unlike Normalized Cross Correlation (NCC), SIFT succeeds at linking corresponding features between the original and geometrically distorted views.
- By leveraging scale-space extrema detection and orientation assignment, the matching links perfectly connect identical structures (such as buildings, walls, and roads) despite the 30-degree rotation and 0.7 scale factor.